

Project Proposal: Trust, but Verify

A Multi-Agent Framework for Auditing LLM-Guided Android Security Analysis

1. Motivation and Context

Android hybrid applications connect native code (Java/Kotlin) with web-based JavaScript components mainly through the WebView framework. The key mechanism behind this is the

`addJavascriptInterface()` function, which exposes Java objects and their methods directly to JavaScript. This is a flexible design, but it also opens up complex cross-language data-flow paths that are not always easy to reason about.

Static analysis tools can identify Java-side interface objects and exposed methods, which gives us a solid starting point for understanding the native side of an app. That said, they run into real trouble when trying to reason about dynamic JavaScript behavior, like code fetched from remote sources or content that is generated on the fly. They also tend to miss higher-level semantics, such as whether a method name or signature hints at sensitive functionality.

Large Language Models (LLMs) look promising here because they can interpret structured code summaries and reason about likely usage patterns. The catch is that LLMs are non-deterministic, so we cannot just take their output at face value. This project proposes a framework where an LLM does the heavy lifting on analysis, but a multi-agent "Auditor" system sits on top of it to measure how much we can actually trust the results.

2. Hypotheses

To evaluate the reliability of this multi-agent verification system, we propose the following statistical tests:

- **Null Hypothesis (H0):** The mean error-detection rate of the best-performing Auditor agent is less than or equal to a baseline random classifier ($p = 0.05$).
- **Alternative Hypothesis (H1):** The mean error-detection rate of the best-performing Auditor agent is significantly greater than the baseline, showing a real and measurable ability to catch hallucinations or logical errors in the primary analysis.

3. Research Questions

We have refined the research questions to focus on how we measure and compare agent quality:

- **RQ A (Quantifying Trustworthiness):** What combination of metrics (such as Semantic Precision, Argumentative Consistency, and False Positive Rate) best captures a meaningful "Trustworthiness Score" for LLM-based security auditing?
- **RQ B (Agent Benchmarking):** Using those trustworthiness metrics, how do different model architectures (GPT, Gemini and different open-weight models) compare when cross-verifying Java-JavaScript data-flow risks?
- **RQ C (Failure Taxonomy):** Where does the Auditor struggle or shine? We are particularly interested in failure modes tied to context window limits or the challenges of reasoning across two different languages.

4. Methodology

The project follows a four-phase workflow that builds on the original static analysis goals while introducing a dedicated verification layer:

Phase	Description	Relation to Original Project
Phase 1: Extraction	Use an existing framework to extract precise Java-side facts about WebView interfaces.	Consistent with original objective.
Phase 2: Analysis	Agent A (Analyzer) interprets the extracted facts to identify potential Java-JavaScript data flows and security risks.	Core of original project.
Phase 3: Verification	Agent B (Auditor) and Agent C (Cross-Checker) review Agent A's output for logical consistency and plausibility.	New Variation.
Phase 4: Evaluation	Qualitative and quantitative reporting on agent performance against a manually verified "Gold Standard."	Evolution of original Phase 4.

5. Expected Outcomes

The project aims to deliver the following:

- A working prototype of an LLM-guided analysis framework with a built-in multi-agent verification layer.
- A clear, standardized set of metrics for calculating a "Trustworthiness Score" for AI-generated code analysis.
- Case studies showing how the Auditor framework catches "plausible but wrong" security findings in hybrid apps.
- A comparative performance report with concrete statistics on which LLM agents hold up best for automated quality assurance.